

Exercise 4: Save a User Selection

Interfaces 102: Building Dynamic Interfaces

Introduction	2
How Can I Practice?	2
Appian Version	2
Naming Conventions	2
Save Often	2
Additional Resources	2
Add a!dynamicLink() to Your Cards	3
Add Dynamic Styling to Your Cards	4
Example Interface Expression	7

Notice of Rights

This document was created by Appian Corporation, 7950 Jones Branch Dr, Tysons, Virginia 22102. Copyright 2026 by Appian Corporation. All rights reserved. Information in this document is subject to change. No part of this document may be reproduced or transmitted in any form by any means, without prior written permission of Appian Corporation. For more information on obtaining permission for reprints or excerpts, contact Appian Training at academyonline@appian.com.

Introduction

How Can I Practice?

You should practice in Appian Community Edition. Keep in mind that this is a community environment not suitable for production workloads or sensitive information.

Appian Version

Appian Community Edition is on the latest Appian version. If you are following the exercises from a previous Appian version, go to academy.appian.com to download the latest version.

Naming Conventions

When you register for Appian Community Edition, you gain access to a workspace, which is your personal area within the community environment. When your workspace has been assigned, you will receive a confirmation email with a workspace ID. Pre-loaded apps in your workspace include the workspace ID.

Use this workspace ID when creating new applications and objects. Otherwise, you could run into naming conflicts with objects created by other users. In the example below, the application prefix for the Acme Auto Solution app is W0000AS. So, the workspace ID is W0000, which would be included in any new app's prefix.

Save Often

Appian does not automatically save updates, so save your objects frequently.

Additional Resources

Appian provides a number of training resources for Appian developers. The following resources are particularly popular with our learners:

- [Academy Online](#) - Appian's online courses provide useful survey courses, step-by-step tutorials, and practice exercises. Explore these resources at your own pace. Survey courses will help you develop a better grasp of the topics you need to learn. Video and print tutorials will help you with getting hands-on experience with Appian.
- [Community Discussions for New Users](#) - Check out the **New to Appian** thread in Community. Join our community of experts to ask questions and find answers from past discussions.
- [Appian Documentation](#) - Appian's product documentation will provide you with an overview of key Appian features, newest release information, additional tutorials, and helpful patterns and recipes to implement in your app.

Add a!dynamicLink() to Your Cards

In the last exercise, you added cards to your interface dynamically using a!forEach(). In this exercise you will add links to those cards to store a user selection and configure dynamic styling to indicate the selected card to the user.

1. Find the **a!cardLayout()** within the **a!forEach()** function and add the parameter **link**.

```
cards: a!forEach(  
  items: local!availableMaintenanceRequests,  
  expression: a!cardLayout(  
    padding: "NONE",  
    link:   
    contents: {↔}  
  )  
)
```

Tip: Remember you can click elements in the live preview to navigate to them in the interface expression. Although this does not work to specifically target elements created dynamically (the card layouts) you can easily navigate to the parent **a!cardGroupLayout()** and locate the child cards from there.

2. Add the function **a!dynamicLink()** to the link parameter. Then specify the parameters within **a!dynamicLink()** as follows:
 - **value:** fv!item[W#AA Maintenance.id]
 - **saveInto:** ri!estimate[W#ADX Estimate.maintenanceId]

```
expression: a!cardLayout(  
  padding: "NONE",  
  link: a!dynamicLink(  
    value: fv!item[W5656AA01 Maintenance.id],  
    saveInto: ri!estimate[W5656ADX Estimate.maintenanceId]  
  ),  
  contents: {↔}  
)
```

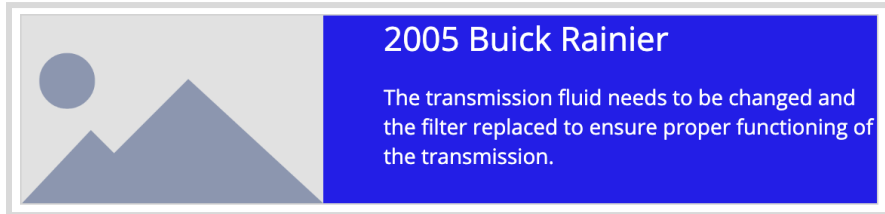
3. Test that the link is working currently by clicking on a card in the interface and checking in the **Rule Inputs** pane that the rule input is updated with the id of the selected maintenance request.

▼ Rule Inputs +	
Name	Value
☒ estimate	[W5656ADX Estim...
☒ maintenanc...	22
☐ cancel	null

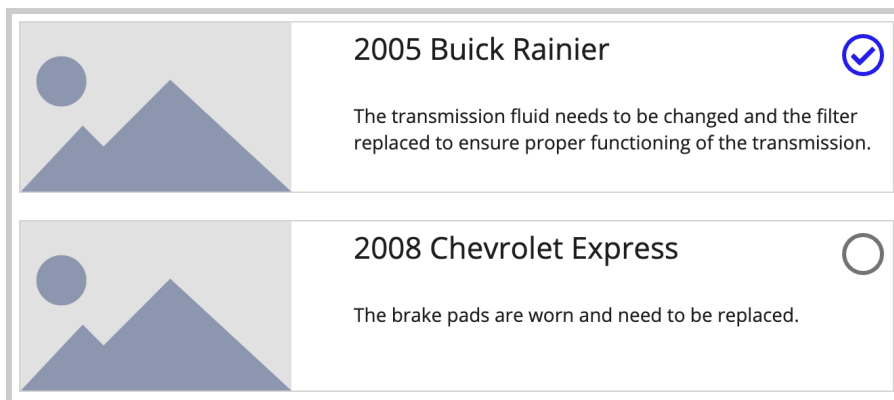
4. Save your changes.

Add Dynamic Styling to Your Cards

Next, you will add dynamic styling to the cards in your interface so that the selected card uses the style ACCENT.



Changing the style of the card is a simple way to show that it has been selected, but the same technique could be used to designate the selection in other ways. For example, dynamically displaying a different icon for selected versus unselected cards.



1. Add the style parameter to your **a!cardLayout()** function.

```
expression: a!cardLayout(  
  padding: "NONE",  
  link: a!dynamicLink(↔),  
  style: ↔,  
  contents: {↔}  
)
```

2. Add an **a!match()** function to the style parameter and configure the parameters as follows:
 - **value:** `ri!estimate[W#ADX Estimate.maintenanceId]`

- **default: "NONE"**

```
expression: a!cardLayout(
  padding: "NONE",
  link: a!dynamicLink(↔),
  style: a!match(
    value: ri!estimate [ W5656ADX Estimate.maintenanceId ],
    default: "NONE"
  ),
  contents: {↔}
)
```

The **a!match()** function allows you to specify a series of conditions paired with expressions to evaluate when those conditions are met. If no conditions are met, **a!match()** will evaluate the **default** parameter. Since no conditions were specified here, the function will always use the default parameter ("**NONE**" in this case) so your cards should still have no styling applied.

3. Add a new line after the **value** parameter, and add a the following parameters:

- **equals: fv!item[W#AA Maintenance.id]**
- **then: "ACCENT"**

```
expression: a!cardLayout(
  padding: "NONE",
  link: a!dynamicLink(↔),
  style: a!match(
    value: ri!estimate [ W5656ADX Estimate.maintenanceId ]
    equals: fv!item [ W5656AA01 Maintenance.id ],
    then: "ACCENT",
    default: "NONE"
  ),
  contents: {↔}
)
```

You have just added a condition to the **a!match()** function. When the **value** parameter is the same as the **equals** parameter, the function will evaluate the **then** parameter. In this case, when the maintenanceId field in the rule input matches the id of the current maintenance request, the card's style will be "**ACCENT**".

4. Test that clicking a card in the interface updates that card's style.
5. Save your changes.

Note: You might be wondering why **a!match()** was chosen instead of **if()**. Indeed, the two expressions below produce very similar behavior:

```
if(
  condition: fv!item[W#AA Maintenance.id] =
    ri!estimate[W#ADX Estimate.maintenanceId],
  valueIfTrue: "ACCENT",
  valueIfFalse: "NONE"
)

a!match(
  value: ri!estimate[W#ADX Estimate.maintenanceId],
  equals: fv!item[W#AA Maintenance.id'],
  then: "ACCENT",
  default: "NONE"
)
```

The difference arises when the rule input is null (as it would be when the user first opens the form). In that case, using the **if()** function would result in an error as, in the **condition** parameter, it is trying to compare values of two different types (null and integer). There are ways to resolve this (by casting or using the **and()** function), but using **a!match()** does not raise this issue and is, in general, more robust.

Example Interface Expression

A full example expression of what your interface might look like at this stage is available below. Keep in mind that you will not be able to copy and paste this into your interface as record type field references are unique to each workspace.

```
a!localVariables(  
  local!availableMaintenanceRequests:  
    rule!W#ADX_getRequestedMaintenance(),  
  a!wizardLayout(  
    titleBar: a!headerTemplateFull(  
      title: "Create Estimate",  
      secondaryText: "Enter details for the estimate"  
    ),  
    isTitleBarFixed: true,  
    showButtonDivider: true,  
    backgroundColor: "#F6F7F9",  
    steps: {  
      a!wizardStep(  
        label: "Details",  
        contents: {  
          a!cardLayout(  
            contents: {  
              a!cardGroupLayout(  
                cards: a!forEach(  
                  items: local!availableMaintenanceRequests,  
                  expression: a!cardLayout(  
                    padding: "NONE",  
                    link: a!dynamicLink(  
                      value: fv!item[W#AA Maintenance.id],  
                      saveInto: ri!estimate[W#ADX Estimate.maintenanceId]  
                    ),  
                    style: a!match(  
                      value: ri!estimate[W#ADX Estimate.maintenanceId],  
                      equals: fv!item[W#AA Maintenance.id],  
                      then: "ACCENT",  
                      default: "NONE"  
                    ),  
                    contents: {  
                      a!columnsLayout(  
                        columns: {  
                          a!columnLayout(  
                            width: "NARROW",  
                            contents: a!imageField(  
                              labelPosition: "COLLAPSED",  
                              images: a!documentImage(  
                                document: if(  
                                  condition: a!isNullOrEmpty(  
                                    fv![W#AA  
                                      Maintenance.vehicle.image]  
                                  ),  
                                  valueIfTrue:  
                                    a!EXAMPLE_DOCUMENT_IMAGE(),  
                                )  
                              )  
                            )  
                          )  
                        )  
                      )  
                    )  
                  )  
                )  
              )  
            )  
          )  
        )  
      )  
    }  
  )  
)
```

```

        valueIfFalse:
            fv![W#AA
                Maintenance.vehicle.image]
        )
    )
)
),
a!columnLayout(
    contents: {
        a!headingField(
            text: {
                fv!item[W#AA Maintenance.vehicle.year]
                & " " &
                fv!item[W#AA Maintenance.vehicle.make]
                & " " &
                fv!item[W#AA Maintenance.vehicle.model]
            },
            size: "MEDIUM"
        ),
        a!richTextDisplayField(
            labelPosition: "COLLAPSED",
            value: fv!item[W#AA Maintenance.issue]
        )
    }
)
)
)
)
)
)
)
),
shape: "SEMI_ROUNDED",
padding: "MORE",
marginBelow: "STANDARD",
style: "NONE"
)
}
),
a!wizardStep(
    label: "Part",
    contents: {
        a!cardLayout(
            contents: {
                a!gridLayout(
                    label: "Part",
                    labelPosition: "COLLAPSED",
                    instructions: "Enter parts for the estimate",
                    height: "AUTO",
                    borderStyle: "LIGHT",
                    shadeAlternateRows: false,
                    rowHeader: 1,
                    headerCells: {

```

```

        a!gridLayoutHeaderCell(label: "Name"),
        a!gridLayoutHeaderCell(label: "Unit Cost"),
        a!gridLayoutHeaderCell(label: "Quantity"),
        a!gridLayoutHeaderCell()
    },
    columnConfigs: {
        a!gridLayoutColumnConfig(width: "NARROW_PLUS"),
        a!gridLayoutColumnConfig(width: "NARROW_PLUS"),
        a!gridLayoutColumnConfig(width: "NARROW_PLUS"),
        a!gridLayoutColumnConfig(width: "ICON")
    },
    rows: a!forEach(
        items: ri!estimate[W#ADX Estimate.part],
        expression: a!gridRowLayout(
            id: fv!index,
            contents: {
                a!textField(
                    value: fv!item[W#ADX Part.name],
                    saveInto: fv!item[W#ADX Part.name]
                ),
                a!floatingPointField(
                    value: fv!item[W#ADX Part.unitCost],
                    saveInto: fv!item[W#ADX Part.unitCost],
                ),
                a!integerField(
                    value: fv!item[W#ADX Part.quantity],
                    saveInto: fv!item[W#ADX Part.quantity]
                ),
                a!imageField(
                    size: "ICON",
                    images: a!documentImage(
                        document: a!iconIndicator(icon: "REMOVE"),
                        altText: "Remove Row" & char(32) & fv!index + 1,
                        link: a!dynamicLink(
                            value: fv!index,
                            saveInto: {
                                a!save(
                                    ri!estimate[W#ADX Estimate.part],
                                    remove(
                                        ri!estimate[W#ADX Estimate.part],
                                        save!value
                                    )
                                )
                            }
                        )
                    )
                )
            }
        )
    ),
    addRowLink: a!dynamicLink(
        label: "Add Row",
        value: W#ADX Part(),
    )
)

```

```

        saveInto: a!save(
            ri!estimate[W#ADX Estimate.part],
            append(
                ri!estimate[W#ADX Estimate.part],
                save!value
            )
        )
    ),
    shape: "SEMI_ROUNDED",
    padding: "MORE",
    marginBelow: "STANDARD",
    style: "NONE"
)
}
)
},
primaryButtons: a!buttonWidget(
    label: "Create",
    submit: true,
    validate: true,
    style: "SOLID",
    showWhen: fv!isLastStep
),
secondaryButtons: a!buttonWidget(
    label: "Cancel",
    submit: true,
    validate: false,
    saveInto: ri!cancel,
    style: "LINK",
    value: true
),
style: "DOT_VERTICAL",
showTitleBarDivider: false,
contentsWidth: "FULL"
)
)

```